Finite Element Method

Mesh Generation

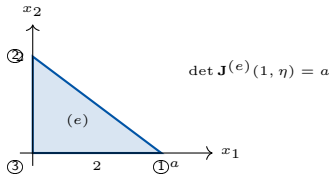
MECH4103 — Spring 2026

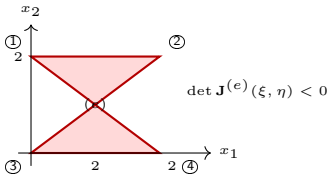
Dr. Behrouz Arash
Associate Professor in Mechanical Engineering

Oslo Metropolitan University

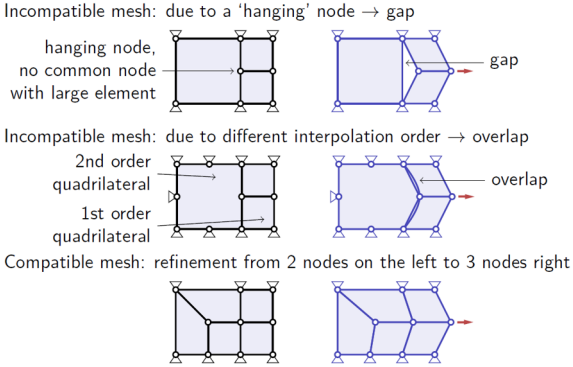
April 14, 2026

- Mesh generation is a critical yet challenging aspect of computational science and engineering, particularly when dealing with 3D geometries.
- Creating a mesh for a geometry involves discretising a continuous domain into smaller elements that a computer can use for simulations. However, several factors make this process difficult:
  - **Geometric Complexity**: Geometries often feature intricate details such as curved surfaces, sharp edges, or internal structures like holes. The mesh must conform to these features without introducing distorted or low-quality elements.
  - **Element Quality**: The quality of mesh elements (e.g. their shape and size) directly affects simulation accuracy. Poorly shaped elements, such as those with extreme angles or bad aspect ratios, can cause numerical instability or inaccurate results.





- **Compatibility Issues**: Meshes must be compatible across different regions of a model, particularly when using multiple subdomains or different element types. Incompatibilities can arise due to:
  - **Hanging Nodes**: When adjacent mesh regions have different element sizes or refinements, nodes may not align, creating gaps or overlaps. This leads to discontinuities in the solution, affecting convergence.
  - **Interpolation Order Mismatch**: If elements with different interpolation orders (e.g. linear vs. quadratic) are adjacent, they may not properly share nodal information, resulting in artificial discontinuities in the numerical solution.



Top: incompatible mesh due to a hanging node → gap. Middle: incompatible mesh due to different interpolation order → overlap. Bottom: compatible refinement from 2 nodes on the left to 3 nodes on the right.

- **Automation and Robustness**: Manually generating meshes for complex 3D geometries is impractical, so automated algorithms are essential. However, these algorithms may fail or produce suboptimal meshes for certain shapes, often requiring manual adjustments.
- **Scalability**: Large-scale simulations, such as those in aerospace or climate modelling, require meshes with millions or even billions of elements. Generating and managing these meshes efficiently is computationally demanding.
- **Adaptivity**: Some simulations need meshes that adapt to solution features, such as refining in areas with steep gradients. This adaptive mesh refinement adds further complexity to the process.
- In the following, we discuss **Gmsh**, an open-source mesh generator. Its design goal is to provide a fast, light and user-friendly meshing tool with parametric input and flexible visualisation capabilities.

What Is a Delaunay Triangulation?

Given a set of points $\mathcal{P} = \{\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_n\}$ in the plane, a **triangulation** is a subdivision of the convex hull of $\mathcal{P}$ into non-overlapping triangles whose vertices belong to $\mathcal{P}$. Among all possible triangulations, the **Delaunay triangulation** $\mathcal{DT}(\mathcal{P})$ is the unique one satisfying the *empty circumcircle property*:

Delaunay criterion (empty circumcircle property)

The circumscribed circle (circumcircle) of every triangle in $\mathcal{DT}(\mathcal{P})$ contains *no other point* of $\mathcal{P}$ in its interior.

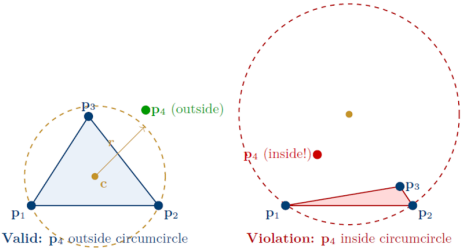
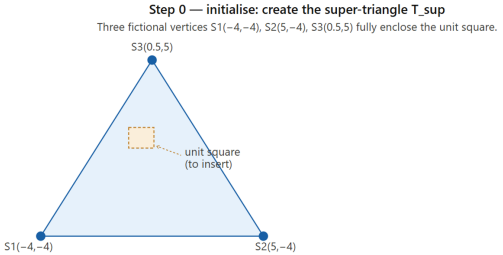


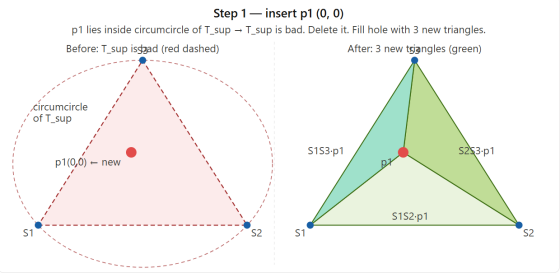
Figure 1: Left: a Delaunay-valid triangle — its circumcircle (dashed gold) contains no other point. Right: a violation — the sliver triangle's circumcircle encloses $\mathbf{p}_4$.

Algorithm 1: Bowyer–Watson Incremental Delaunay Triangulation

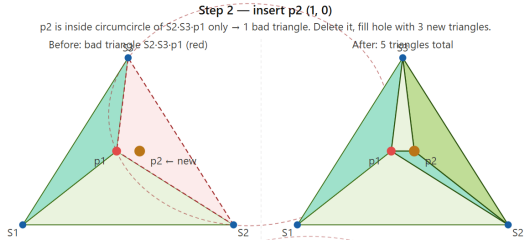
1. **Initialise.** Create a *super-triangle* T_{sup} large enough to contain all input points. Set $\mathcal{T} \leftarrow \{T_{\text{sup}}\}$.
2. **Insert point.** For each new point $\mathbf{p} \in \mathcal{P}$:
   1. *Find bad triangles.* Collect $\mathcal{B} = \{T \in \mathcal{T} : \mathbf{p} \text{ lies inside the circumcircle of } T\}$.
   2. *Find boundary polygon.* Identify the set of edges $\mathcal{E}$ that border $\mathcal{B}$ on exactly one side (i.e. edges shared by exactly one bad triangle). These form a star-shaped polygonal hole.
   3. *Remove bad triangles.* $\mathcal{T} \leftarrow \mathcal{T} \setminus \mathcal{B}$.
   4. *Re-triangulate hole.* For each edge $e \in \mathcal{E}$, form a new triangle connecting e to $\mathbf{p}$ and add it to $\mathcal{T}$.
3. **Clean up.** Remove all triangles sharing a vertex with T_{sup}.
4. **Output.** $\mathcal{T}$ is the Delaunay triangulation of $\mathcal{P}$.



Step 0 — Initialise: create the super-triangle T_{sup}.
Three fictional vertices S1$(-4,-4)$, S2$(5,-4)$, S3$(0.5,5)$ fully enclose the unit square.

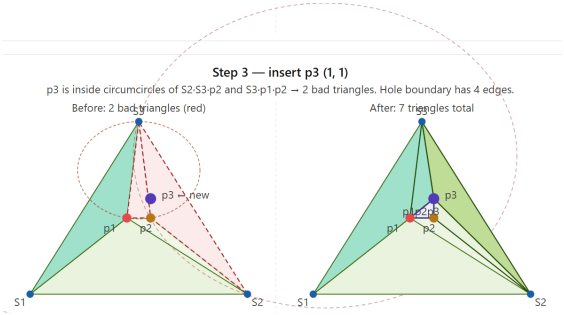


Step 1 — Insert $\mathbf{p}_1(0,0)$.
$\mathbf{p}_1$ lies inside the circumcircle of $T_{\text{sup}} \Rightarrow T_{\text{sup}}$ is bad. Delete it. Fill hole with 3 new triangles.



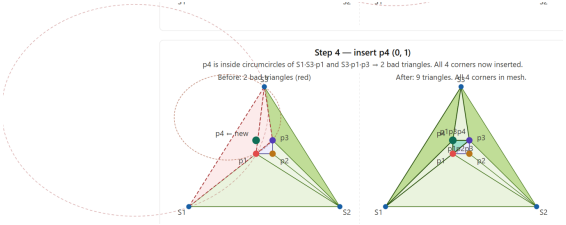
Step 2 — Insert $\mathbf{p_2}(1, 0)$.

$\mathbf{p_2}$ is inside circumcircle of $S_2 \cdot S_3 \cdot \mathbf{p_1}$ only $\Rightarrow$ 1 bad triangle. Delete it, fill hole with 3 new triangles. Total: 5 triangles.



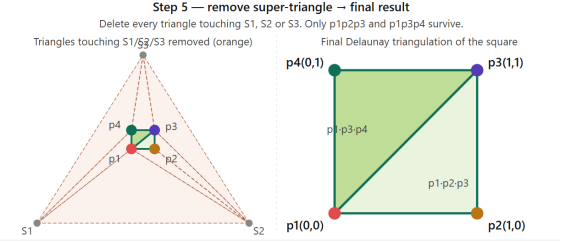
Step 3 — Insert $\mathbf{p_3}(1, 1)$.

$\mathbf{p_3}$ is inside circumcircles of $S_2 \cdot S_3 \cdot \mathbf{p_2}$ and $S_3 \cdot \mathbf{p_1} \cdot \mathbf{p_2}$ $\Rightarrow$ 2 bad triangles. Hole boundary has 4 edges. Total: 7 triangles.



Step 4 — Insert $\mathbf{p}_4(0,1)$.

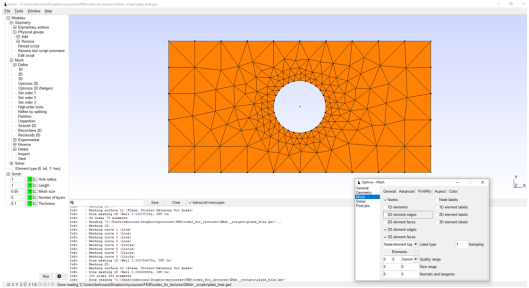
$\mathbf{p}_4$ is inside circumcircles of $S_1 \cdot S_3 \cdot \mathbf{p}_1$ and $S_3 \cdot \mathbf{p}_1 \cdot \mathbf{p}_3 \Rightarrow$ 2 bad triangles. All 4 corners now inserted. Total: 9 triangles.



Step 5 — Remove super-triangle → final result.

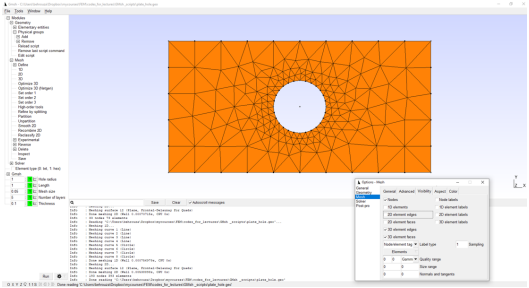
Delete every triangle touching S_1, S_2 or S_3. Only $\mathbf{p}_1\mathbf{p}_2\mathbf{p}_3$ and $\mathbf{p}_1\mathbf{p}_3\mathbf{p}_4$ survive. Final Delaunay triangulation of the square.

- Gmsh (`https://gmsh.info/`) is an open-source 3D finite element grid generator with a built-in CAD engine.
- Handles complex geometries in 1D, 2D, and 3D.
- Supports mesh export in formats like `.msh`, `.stl`, `.m`, and more.
- Provides both a graphical interface and a scripting language for full control.



Gmsh GUI showing the mesh of a plate with a circular hole.

- Gmsh (`https://gmsh.info/`) is an open-source 3D finite element grid generator with a built-in CAD engine.
- Handles complex geometries in 1D, 2D, and 3D.
- Supports mesh export in formats like `.msh`, `.stl`, `.m`, and more.
- Provides both a graphical interface and a scripting language for full control.



Gmsh GUI showing the mesh of a plate with a circular hole.

- Gmsh uses .geo files to describe geometries and meshing operations.
  - These are plain text files written in Gmsh's scripting language.
  - Can be edited manually or generated programmatically (e.g. via Python).
  - Enables precise control over geometry, mesh size, boundary conditions, and physical groups.

A sample Gmsh .geo script

```
lc = 0.1;  // Mesh size
// Define points
Point(1) = {0, 0, 0, lc};
Point(2) = {1, 0, 0, lc};
// Define line
Line(1) = {1, 2};
// Physical group
Physical Line("line") = {1};
// Generate 1D mesh
Mesh 1;
```

Structure of a .geo Script

- **Definition of points**: building blocks of the geometry.
- **Definition of lines and surfaces**: form the shape of the model.
- **Definition of physical groups**: tag boundaries and domains for simulation.
- **Mesh generation commands**: control mesh size, type, and quality.

Point

- Defines a point in 3D space:
  `Point(id) = {x, y, z, meshSize};`
- `meshSize` is optional and sets the local mesh density.

```
Point(1) = {0, 0, 0, lc};
Point(2) = {1, 0, 0, lc};
```

Line and Circle

- `Line(id) = {start, end};` Connects two points.
- `Circle(id) = {start, center, end};` Defines a circular arc via three points.

```
Line(1) = {1, 2};
Circle(2) = {2, 3, 4};
```

Line Loop and Plane Surface

- `Line Loop(id) = {curve1, curve2, ...};` Lists curves to form a closed boundary.
- `Plane Surface(id) = {loop1, loop2, ...};` Creates a 2D surface, with inner loops for holes.

```
Line Loop(10) = {1, 2, 3, 4};
Plane Surface(20) = {10};
```

Physical Groups

- Tags entities (e.g. lines, surfaces) for simulation boundary conditions or domains.
- Essential for linking geometry to simulation software.

```
Physical Surface("domain") = {20};
Physical Line("left") = {1};
```

- The **Field** command in Gmsh is very versatile — it is used to control mesh size fields based on a wide range of criteria such as distance from a boundary.
- This command can do many different things depending on the `Field` type chosen.

Example: Using a Threshold Field to Vary Mesh Size Smoothly

```
Field[2] = Threshold;           // Define a transition zone
Field[2].InField  = 1;          // Base this threshold on Field[1]
Field[2].SizeMin  = lc_fine;    // Fine mesh size in the refined region
Field[2].SizeMax  = lc_coarse;  // Coarser mesh size far from the region
Field[2].DistMin  = R_ref;      // Full refinement up to distance R_ref
Field[2].DistMax  = R_ref * 1.5; // Gradual transition to coarse mesh
```

→ `Field[2]` uses the distance field to smoothly transition from fine to coarse mesh.

- `Mesh.Algorithm` selects the meshing algorithm:

`Mesh.Algorithm = 1` — Adaptive

Generates triangular meshes, refining based on geometry or solution features. Best for complex geometries requiring adaptive refinement.

`Mesh.Algorithm = 2` — Automatic

Gmsh selects an algorithm based on geometry and settings (default/fallback). Can produce triangular or quadrilateral elements.

`Mesh.Algorithm = 3` — Delaunay

Uses Delaunay triangulation to create well-shaped triangles. Best for general-purpose triangular meshes.

`Mesh.Algorithm = 4` — Advancing Front

Starts from boundaries and moves inward to produce high-quality triangular elements near boundaries. Best for fluid dynamics with boundary layers.

`Mesh.Algorithm = 5` — Anisotropic

Specialised algorithm for anisotropic triangular meshes, allowing elements to stretch in specific directions to capture directional features.

`Mesh.Algorithm = 6` — Frontal Quads

Generates unstructured quadrilateral meshes by advancing from boundaries inward. Best when quadrilateral elements are preferred.

`Mesh.Algorithm = 7` — Quads (Structured-like)

Generates quadrilateral meshes aiming for a structured arrangement where possible.

`Mesh.Algorithm = 8` — Quasi-Structured Quads

Produces quasi-structured quadrilateral meshes. Best for nearly rectangular geometries.

- `Mesh.ElementOrder = 2` — Quadratic elements:
  - Increases accuracy by using curved (higher-order) elements.
  - Results in denser meshes with more degrees of freedom.

Structured Mesh

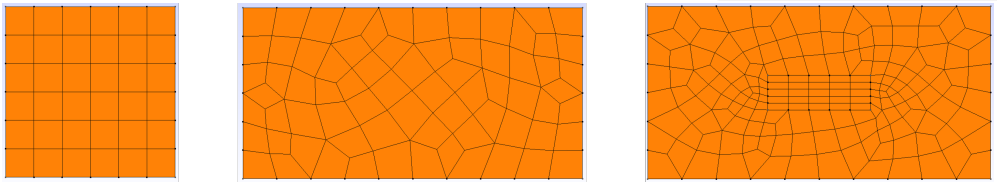
- Elements arranged in a regular, grid-like pattern.
- Nodes and elements follow an orderly numbering sequence.
- Best for simple geometries (rectangles, squares).
- Example: uniform quadrilateral mesh of a rectangular domain.

Unstructured Mesh

- Elements randomly distributed, no regular connectivity.
- Useful for complex geometries where structured grids are difficult.
- Provides flexibility but requires advanced meshing algorithms.
- Example: triangular or quad mesh for an irregular domain.

Quasi-Structured Mesh

- Mix of structured and unstructured regions.
- Local structured patterns transitioning into unstructured zones.
- Useful for domains with complex boundaries but large structured regions.
- Example: quads structured in the interior, unstructured near boundaries.



Left: structured mesh. Centre: unstructured mesh. Right: quasi-structured mesh.

1D Example: Mesh along a Line from $(0, 0, 0)$ to $(1, 0, 0)$

```
lc = 0.1;  // Mesh size
// Define points
Point(1) = {0, 0, 0, lc};
Point(2) = {1, 0, 0, lc};
// Define line
Line(1) = {1, 2};
// Physical group
Physical Line("line") = {1};
// Generate 1D mesh
Mesh 1;
```

Using the Gmsh GUI

1. Launch Gmsh.
2. **File** → **Open...** (or Ctrl+O) and select `example_1D.geo`.
3. View the geometry (defined points and line).
4. **Mesh** → **1D** to generate the mesh.
5. Inspect the mesh using the mouse (zoom/pan).

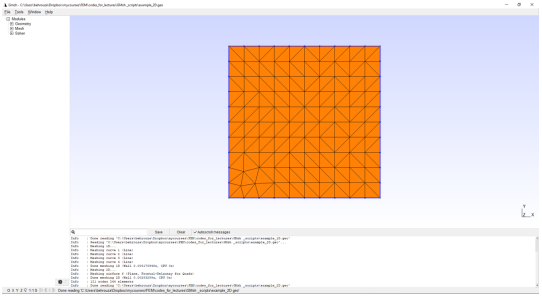
Using the Command Line

1. Navigate to the directory containing `example_1D.geo`.
2. Run: `gmsh example_1D.geo -1`

2D Example: Square Mesh

```
lc = 0.1;  // Mesh size
// Define square corners
Point(1) = {0, 0, 0, lc};
Point(2) = {1, 0, 0, lc};
Point(3) = {1, 1, 0, lc};
Point(4) = {0, 1, 0, lc};
// Define lines
Line(1) = {1, 2};
Line(2) = {2, 3};
Line(3) = {3, 4};
Line(4) = {4, 1};
// Line loop and surface
Line Loop(5) = {1, 2, 3, 4};
Plane Surface(6) = {5};
// Physical groups
Physical Surface("square") = {6};
Physical Line("bottom") = {1};
Physical Line("right")  = {2};
Physical Line("top")    = {3};
Physical Line("left")   = {4};
// Generate 2D mesh
Mesh 2;
```

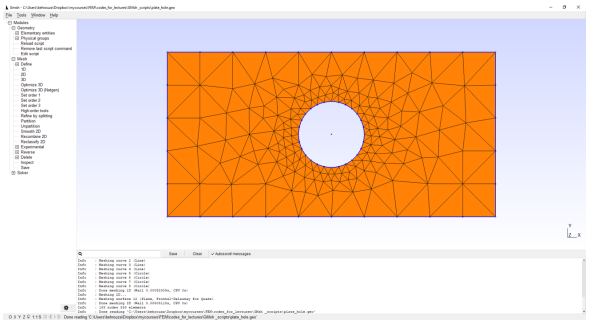
- Generates a mesh for a simple 1×1 square with labelled boundaries.



Gmsh output: triangulated 1×1 square mesh with labelled boundary edges.

- The Gmsh script is available on Canvas.

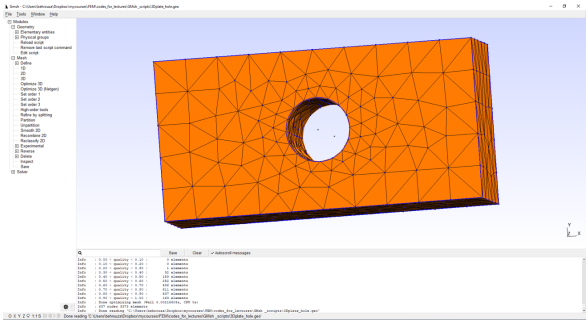
2D Example: Plate with a Circular Hole



Gmsh output: triangulated plate with a circular hole; finer mesh near the hole boundary using a distance field.

- Uses `Circle` arcs to define the hole boundary.
- The hole is subtracted from the plate surface using nested `Line Loops` in `Plane Surface`.
- A `Threshold` field refines the mesh near the hole.
- The Gmsh script is available on Canvas.

3D Example: Extruded Plate with a Circular Hole



Gmsh output: 3D tetrahedral mesh of a plate with a circular hole, generated by extruding the 2D surface mesh.

- The 2D surface mesh is extruded in the thickness direction using the `Extrude` command.
- Physical volumes and surfaces are tagged for boundary condition assignment.
- Mesh quality statistics (element quality histogram) are reported by Gmsh after meshing.
- The Gmsh script is available on Canvas.

Using the Gmsh GUI

1. Generate the mesh from your `.geo` file.
2. Press **F11** to switch to mesh view.
3. **File** → **Export...**
   - `.msh`: select *Gmsh Mesh Files (*.msh)*.
   - `.m`: select *Medit (*.m)*.
4. Click **Save**.

Using the Command Line

Generate and export `.msh`:

```
gmsh example_1D.geo -2 -o example_1D.msh
```

Generate and export `.m` (Medit format):

```
gmsh example_1D.geo -2 -format med -o example_1D.m
```

Next: FEM for Multidimensional Vector Field Problems – Part 1

Thanks for your attention!